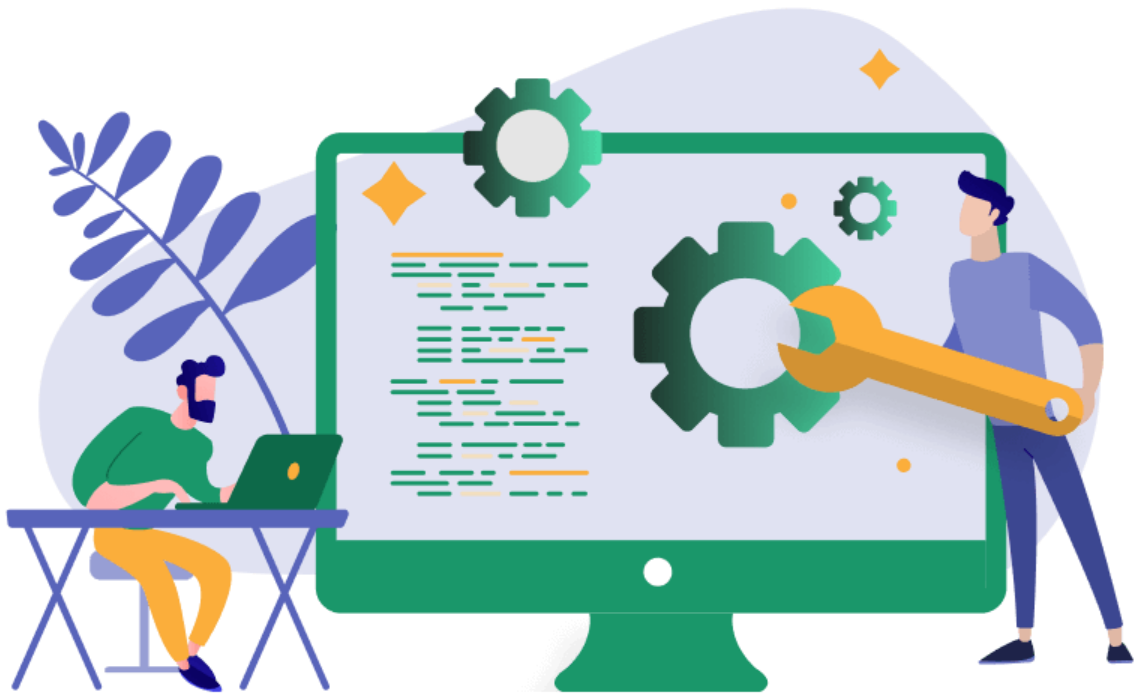




Name:

Muhammad Madni

Wasif Ali Raza

Muhammad Daud Latif

Zohaib Luqman

Haneeza Awan

Submitted to Dr. Nabeel Ahmed

Software Requirements Specification (SRS)

Threat Intelligence System

1. Introduction

1.1 Purpose

The purpose of this project is to design and develop a **Threat Intelligence System** that collects, stores, and manages Indicators of Compromise (IOC) such as IP addresses, domains, severity levels, and sources. The system helps in identifying and analyzing potential cybersecurity threats.

1.2 Scope

This system is a web-based application built using Spring Boot, MySQL, and Kafka. It allows users to add and view threat data through a simple dashboard and provides backend APIs for data processing and storage.

1.3 Definitions

- **IOC (Indicator of Compromise):** Evidence of malicious activity such as IP, domain, etc.
- **Kafka:** Messaging system for real-time data streaming.
- **Spring Boot:** Backend framework for Java applications.

1.4 Technologies Used

- Java 17
- Spring Boot
- Spring Data JPA
- MySQL
- Apache Kafka
- HTML, JavaScript, CSS
- Maven

2. Overall Description

2.1 Product Perspective

The system is a standalone web application that integrates:

- Frontend dashboard
- Backend REST APIs
- Database (MySQL)
- Messaging system (Kafka)

2.2 Product Functions

The system performs the following functions:

- Add IOC data (IP, domain, severity, source)
- Store data in MySQL database
- Retrieve all IOC records
- Send data to Kafka topic
- Display data on web dashboard

2.3 User Classes

- System Administrator (manages data)
- Analyst (views threat data)

2.4 Constraints

- System requires MySQL database
- Kafka must be running for messaging
- Java 17 or above required
- Local server environment (localhost)

2.5 Assumptions

- Users have basic understanding of threat data
- Kafka and MySQL services are properly configured
- System runs in a local environment

3. System Features

3.1 Add IOC Data

Description:

User can add threat data through API or dashboard.

Input:

- IP Address
- Domain
- Severity
- Source

Output:

- Stored record in database

3.2 View IOC Data

Description:

Retrieve all stored threat data.

Output:

- List of IOC records

3.3 Kafka Messaging

Description:

System sends IOC data to Kafka topic for processing.

Topic:

- threat-topic

3.4 Web Dashboard

Description:

Simple UI to:

- Add IOC
- View IOC table

4. Functional Requirements

- System must allow adding IOC data
- System must store data in MySQL
- System must retrieve all IOC records
- System must expose REST API endpoints
- System must send messages to Kafka topic
- System must display data on frontend

5. Non-Functional Requirements

5.1 Performance

- System should handle multiple IOC records efficiently

5.2 Security

- Basic validation of input data

5.3 Reliability

- System should ensure data is not lost in database or Kafka

5.4 Scalability

- Kafka allows future scaling for real-time processing

5.5 Usability

- Simple and easy-to-use dashboard interface

6. System Architecture

Frontend (HTML Dashboard)



Spring Boot REST API



Service Layer



Repository Layer



MySQL Database

Kafka Producer → Kafka Topic → Kafka Consumer

7. Database Design

Table: ioc_data

Field	Type	Description
id	Long	Primary Key
ip	String	IP Address
domain	String	Domain Name
severity	int	Threat Severity
source	String	Data Source

8. API Design

8.1 Add IOC

POST /ioc

8.2 Get All IOC

GET /ioc

9. Hardware Requirements

- Processor: Intel i3 or higher
- RAM: 4GB minimum
- Storage: 500MB+ free space

10. Software Requirements

- Windows / Linux OS

- Java JDK 17+
- MySQL Server
- Apache Kafka
- Maven

11. Limitations

- No authentication system
- Basic UI design
- Runs locally only
- No advanced threat analysis (AI/ML not included)

12. Future Enhancements

- Real-time dashboard updates (WebSocket)
- AI-based threat classification
- Authentication & authorization (JWT)
- Cloud deployment
- Advanced analytics dashboard
- Alert system for high severity threats

13. Conclusion

The Threat Intelligence System successfully provides a basic platform for managing and analyzing cybersecurity threats. It integrates backend services, database management, messaging systems, and a simple frontend dashboard, making it a complete full-stack academic project.